

Microdown TOC

Challenge C3P - Université de Lille

Master : MIAGE

Année : M1

GitHub : https://github.com/TokySandratra/MicrodownToc_GROUP10

Présenté par :

- Toky Sandratra Miharimamy RASAMOELINA
- Brunine Einstein POINTE-JOUR
- Marie Changlais AIMÉ

Plan de la présentation

1. Introduction au microdown & objectifs du challenge
2. Design pattern : Composite et visitor
3. Implémentation de la solution
4. Cas limites / Tests
5. Démonstration live et résultats
6. Conclusion

Introduction Microdown

Microdown : Un parser Markdown pour Pharo

Qu'est-ce que Microdown ?

- Librairie Smalltalk / Pharo
- Créer un AST (Abstract Syntax Tree) représentant la structure du document
- Utilisée pour générer de la documentation, des blogs, des contenus enrichis

✗ Besoins non satisfaits :

- Extraire automatiquement tous les titres d'un document Markdown
- Générer une TOC structurée avec niveaux (h1, h2, h3...)
- Créer des liens / anchors vers chaque section
- Filtrer les titres par niveau (ex: afficher que h2 et h3)
- Formater la TOC en différents formats (texte, HTML, Microdown)

Objectif : fournir un format simple, extensible et intégré pour toute la documentation Pharo

Objectifs du Challenge

5 nouvelles fonctionnalités :

1. **TOC texte** : générer une table des matières indentée
2. **Filtrage des niveaux de titres** : showOnlyAbove: pour ne garder que certains niveaux
3. **Numérotation automatique du TOC**: format "1 Microdown", "2 Architecture"
4. **Création d'un mini-document Microdown pour la TOC** : mini-document avec "# Microdown", "# Architecture"
5. **TOC HTML avec liens** : `<a href="#microdown">Microdown</a>`

Design Patterns : Composite

Définition

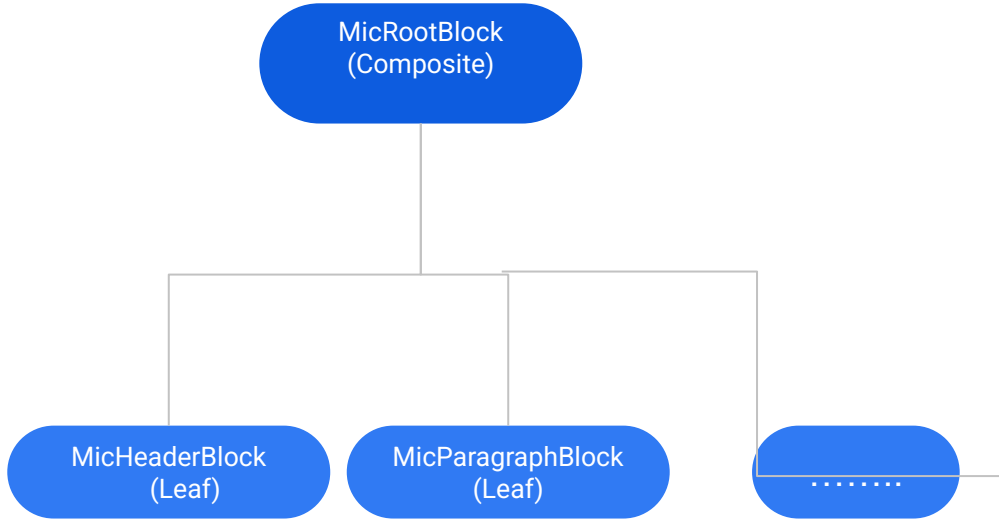
Le pattern Composite permet de représenter une hiérarchie d'objets sous forme d'arbre.

Chaque nœud peut être un élément simple (feuille) ou un conteneur (composite) de plusieurs enfants.

Application dans Microdown

- Le document Markdown est modélisé comme un arbre (AST) de blocs :
 - **MicRootBlock** : racine, contient tous les blocs du document
 - **MicHeaderBlock** : bloc titre (feuille)
 - **MicParagraphBlock** : bloc paragraphe (feuille), peut contenir du texte
 - **MicTextBlock** : bloc texte (feuille)
- Chaque bloc possède une méthode children (sauf les feuilles) pour accéder à ses sous-blocs.

Design Pattern : Composite



MicRootBlock est la racine de l'arbre, il peut contenir des enfants (blocs)

```
MicRootBlock >>  
hasMetaDataElement [  
  ^ children  
  isEmpty: [false]  
  ifNotEmpty: [ children first  
isMetaData ]  
]
```

Accès au premier enfant (composite)

```
MicRootBlock >> metaDataElement [  
  ^ children first  
]
```

Implémentation du Design pattern visitor

- Comment microdown implémente le design pattern visitor
 - Le design pattern visitor permet de séparer la logique métier de la structure des objet.
- L'intérêt de visitor dans ce contexte
- Comment nous avons tiré parti de ce design pattern pour atteindre notre objectif



Implémentation de la solution

Reverse engineering

Microdown



Visitor



MicrodownVisitor



visitHeader: visit: ...

Microdown-Tests



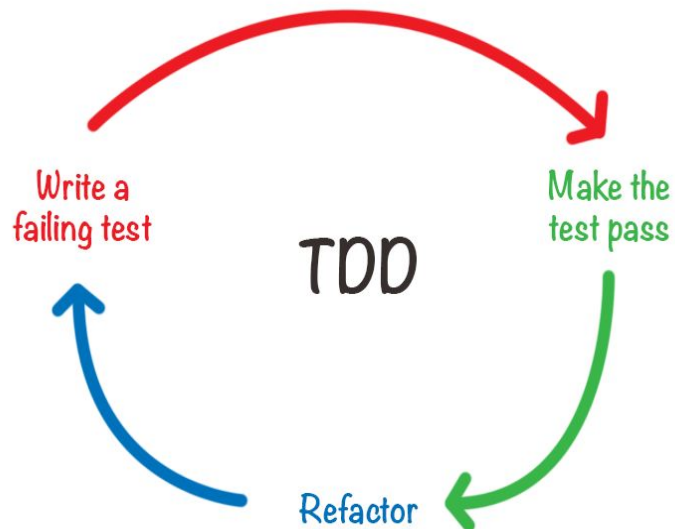
Visitor



MicrodownVisitorTest

Implémentation de la solution

Test-Driven-Development



Tests créés en TDD :

- `testTOCWithoutFilter` (extraction basique)
- `testTOCWithFilter` (ajout du filtrage)
- `testNumberedWithFilter` (numérotation)
- `testMicrodownTOC` (sortie Microdown)
- `testHtmlTOCWithLinks` (liens HTML)

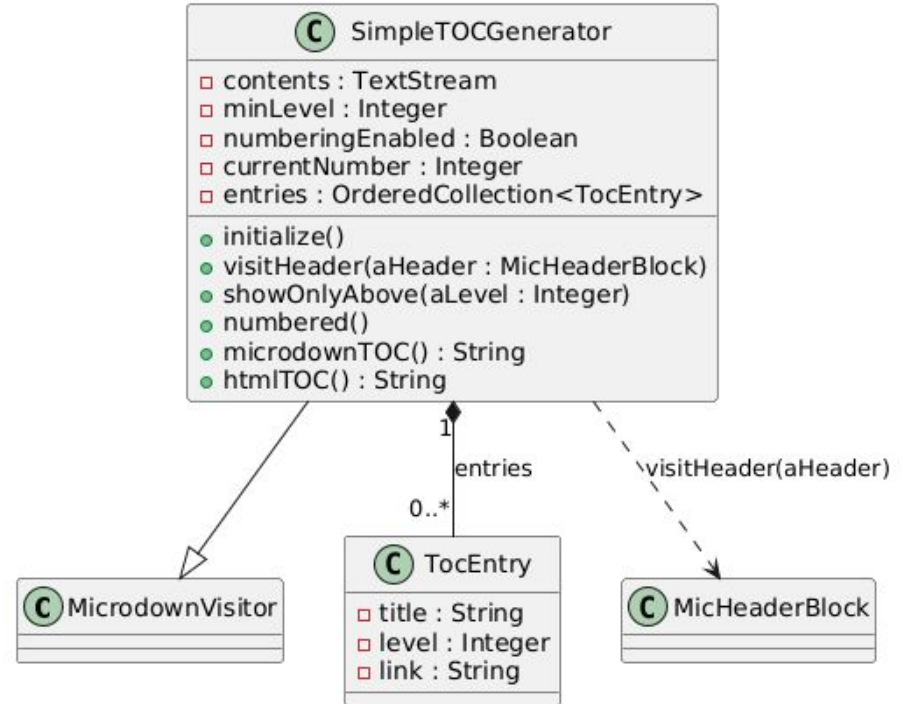
Implémentation de la solution (Design Patterns Utilisés)

Citation importante :

"On a réutilisé l'architecture existante au lieu de réinventer.

Zéro modification du code de base."

Preuve : Nous utilisons aHeader header (redéfinition) au lieu de réinventer.



Démonstration et résultats

The screenshot shows a web-based code playground titled "Playground". The interface is divided into several sections:

- Top Bar:** Contains icons for "Do it all", "Publish", "Bindings", "Versions", and "Pages". On the right, it says "a ByteString (Microdown Ar..." and has icons for a color picker, a green circle, a refresh button, a zoom in/out button, and a close button.
- Code Editor:** On the left, it contains a code editor with the following text:

```
1 miniDoc := '# Microdown
2 # Architecture
3 ## Visitors
4 ## A builder'.
5
6 vis := SimpleTOCGenerator new.
7 vis visit: (Microdown parse: miniDoc).
8 vis contents."doit afficher un TOC avec indentation"
9
10 |
11
12
```

Line 10:1 is highlighted. A "+L" button is at the bottom right of the editor.
- Preview Panel:** On the right, there are tabs for "Preview", "Items", "Full Content", "Encoding", "Raw", "Breakpoints", and "Meta". The "Preview" tab is selected, showing the rendered output of the code:

```
Microdown
Architecture
Visitors
A builder
```
- Bottom Panel:** Below the preview, there is a small section with a tab labeled "1" and the text "self".

Démonstration et résultats

The screenshot shows a web-based code playground titled "Playground". The interface is divided into two main sections: a code editor on the left and a preview area on the right.

Code Editor: The editor contains the following code:

```
1 miniDoc := '# Microdown
2 # Architecture
3 ## Visitors
4 ## A builder'.
5
6 vis2 := SimpleTOCGenerator new.
7 vis2 showOnlyAbove: 1.
8 vis2 visit: (Microdown parse: miniDoc).
9 vis2 contents "ne montre que les titres de niveau >
10 1"
11
12
```

Below the code editor, the status bar shows "Line: 1:1" and a "+L" button.

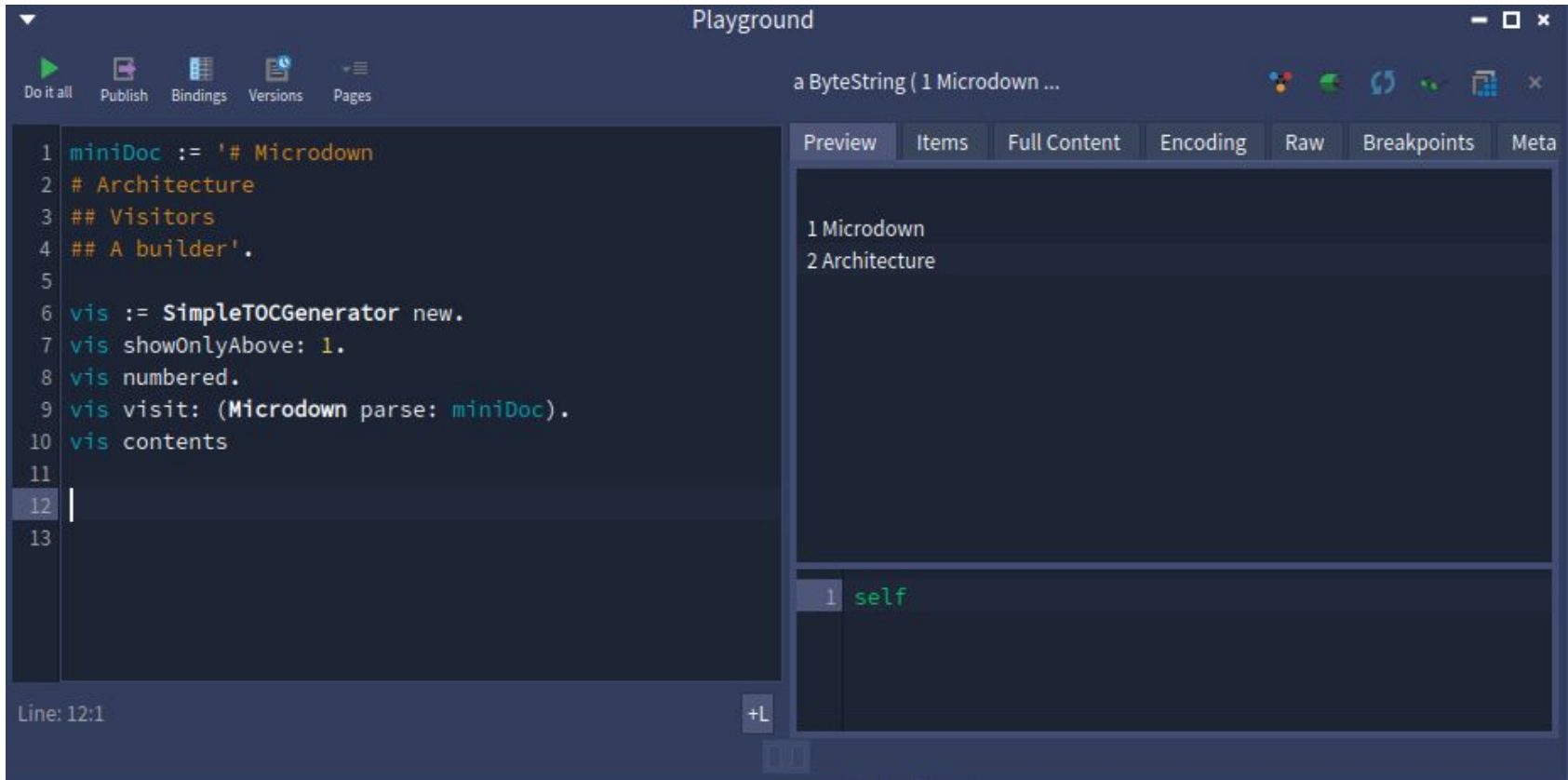
Preview Area: The title bar of the preview area reads "a ByteString (Microdown Ar...". It has tabs for "Preview", "Items", "Full Content", "Encoding", "Raw", "Breakpoints", and "Meta". The "Preview" tab is selected, showing the rendered output of the code:

Microdown
Architecture

Below the preview, there is a small table with one row:

1	self
---	------

Démonstration et résultats



The screenshot shows the Playground application interface. The top bar includes a 'Playground' title and standard window controls. Below the title bar is a toolbar with icons for 'Do it all', 'Publish', 'Bindings', 'Versions', and 'Pages'. The main area is split into two panes. The left pane is a code editor with a dark theme, showing a Scala-like code snippet. The right pane is a preview window with tabs for 'Preview', 'Items', 'Full Content', 'Encoding', 'Raw', 'Breakpoints', and 'Meta'. The 'Preview' tab is active, displaying a rendered document structure. The code in the left pane defines a `miniDoc` string and a `vis` object using `SimpleTOCGenerator`. The preview window shows the resulting document structure, including a list of items and a `self` reference.

```
1 miniDoc := '# Microdown
2 # Architecture
3 ## Visitors
4 ## A builder'.
5
6 vis := SimpleTOCGenerator new.
7 vis showOnlyAbove: 1.
8 vis numbered.
9 vis visit: (Microdown parse: miniDoc).
10 vis contents
11
12
13
```

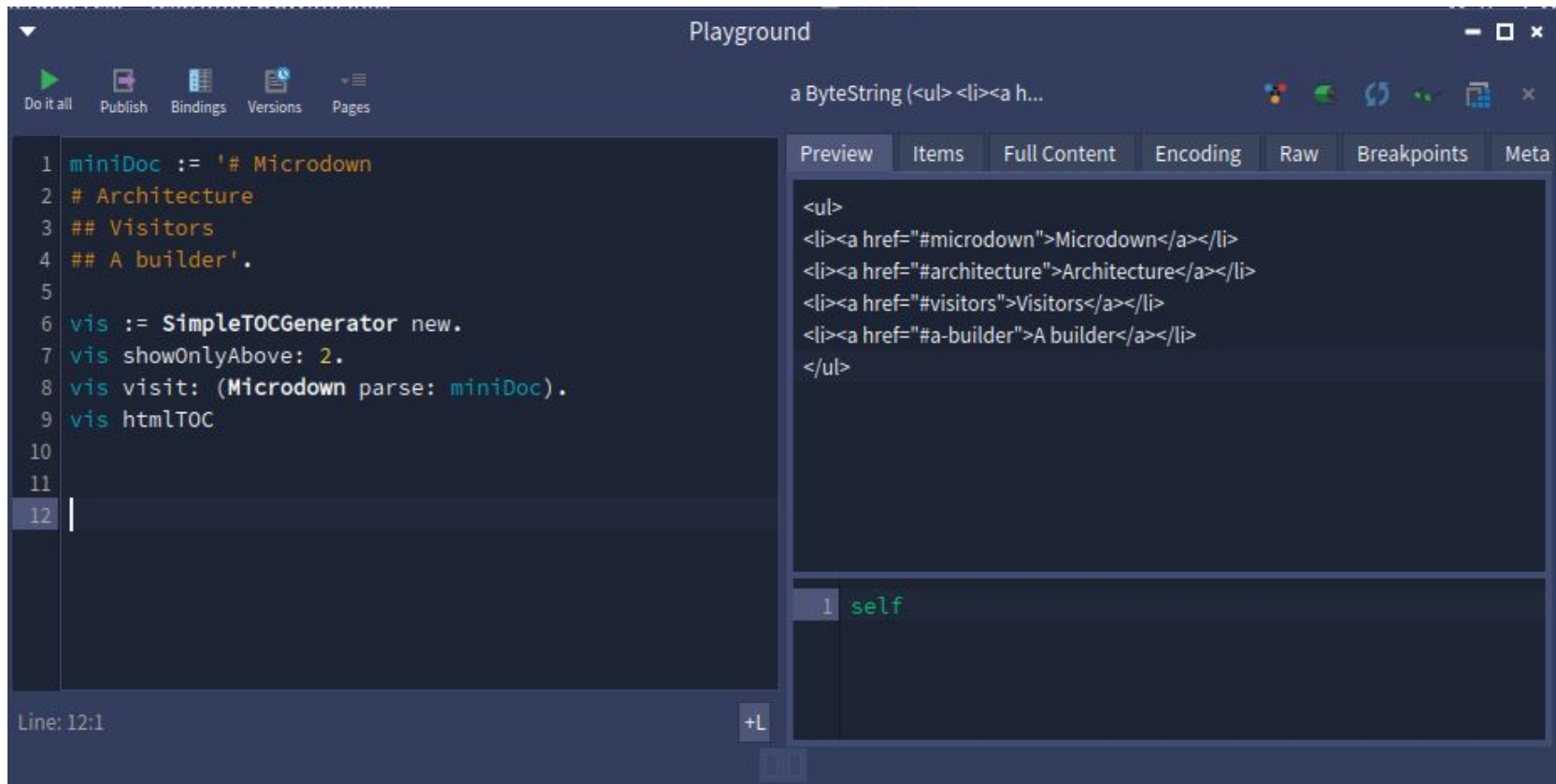
Line: 12:1

Preview

1 Microdown
2 Architecture

1 self

Démonstration et résultats



The screenshot shows a code editor interface with a dark theme. The main editor on the left contains Scala code for a simple TOC generator. The right side features a preview window showing the rendered HTML output. The code defines a `miniDoc` string, a `SimpleTOCGenerator` instance, and a `visit` method that generates an HTML list from the document structure.

```
1 miniDoc := '# Microdown
2 # Architecture
3 ## Visitors
4 ## A builder'.
5
6 vis := SimpleTOCGenerator new.
7 vis showOnlyAbove: 2.
8 vis visit: (Microdown parse: miniDoc).
9 vis htmlTOC
10
11
12 |
```

Line: 12:1

Preview window content:

```
<ul>
<li><a href="#microdown">Microdown</a></li>
<li><a href="#architecture">Architecture</a></li>
<li><a href="#visitors">Visitors</a></li>
<li><a href="#a-builder">A builder</a></li>
</ul>
```

Preview window tabs: Preview, Items, Full Content, Encoding, Raw, Breakpoints, Meta

Conclusion & Perspectives

Ce qu'on a livré :

- 5 fonctionnalités complètes
- 2 design patterns appliqués proprement
- 5 tests unitaires verts
- Code extensible (Open-Closed Principle)

QUESTIONS

